AI Assignment - Reasoning Agent Report

Name: Omkar Deepak Mane

Roll No.: M26CPS005

1. Overview

My system answers True or False for a query, given a knowledge base of facts and rules. It reaches about **92% accuracy on 4,029 public questions** and never calls a language model: every answer comes out of a classical reasoning engine.

The idea is simple. First, I turn the input into a clean internal form, then a router sends it to one of three engines:

1. **CounterBench-style** cause-and-effect questions go to a search engine that tries every possible world.
2. **Logic** (first-order formulas in any common notation) goes to a FOL engine. It uses forward chaining when the rules are simple, and resolution with DPLL when they are not.
3. **English facts and if-then rules** (RuleTaker style) go to a forward-chaining engine with unification.

If no engine can read a question, the system answers False. Each engine is wrapped so that one broken question cannot stop the run.

2. Chosen representation

I use a different representation for each kind of input, because each one fits its reasoning method best. Before that, a small input reader cleans every question into one standard form.

Input reader. The private test is given as logic, but the exact notation was not announced. So the reader accepts all the common ones and converts them to a single set of symbols ($\forall\ \exists\ \neg\ \wedge\ \vee\ \rightarrow\ \leftrightarrow\ \oplus$):

Style	Example	After cleaning
Unicode FOL	`∀x (Young(x) → Smart(x))`	unchanged
ASCII	`forall x (Young(x) -> Smart(x))`	`∀x (Young(x) → Smart(x))`
Prover9	`all x (Young(x) -> -Old(x)).`	`∀x (Young(x) → ¬Old(x))`
Prolog	`smart(X) :- young(X), kind(X).`	`∀v ((young(v) ∧ kind(v)) → smart(v))`

It also removes headers such as `Facts:` and numbering such as `1.`, moves a `Query:` line out of the context, and treats `Bob`, `bob` and `is_young` / `IsYoung` as the same name. The test file itself can be CSV, TSV, JSON, JSONL or plain text; column names are detected automatically.

Logic input → formula trees. Each formula is parsed into a small tree, for example `("all", "x", ("imp", Young(x), Smart(x)))`. An atom is a predicate name plus a tuple of arguments. This is the main representation, since the evaluation input is logic.

English input → facts and rule templates. A fact is a short lower-case sentence such as `bob is young`. A rule is a list of conditions and one conclusion. Words like *someone*, *something*, *they* and *it* become a variable `?x`, so *"If someone is young, then they are smart"* becomes `?x is young` ⇒ `?x is smart`. Shortcuts are expanded: *"smart and kind"* becomes two conditions, and *"All young people are smart"* becomes the same kind of rule.

Cause-and-effect input → structural equations. For CounterBench I store, for every variable, which variables cause it, whether each cause is negated, and whether the causes combine with AND or OR. Observed values and Bernoulli priors are stored separately.

3. Reasoning algorithms used

All three engines use methods from the course: forward chaining, unification, search, and resolution by refutation. None of them learns from data; the knowledge comes only from the facts and rules inside each question.

Engine A – English rules: forward chaining with unification. I start from the known facts and keep firing every rule whose conditions hold, adding its conclusion as a new fact. A condition with a variable is unified with the facts by pattern matching, so `?x is young` matches `bob is young` with `?x = bob`. The loop stops when a full pass adds nothing new, which means there is no fixed depth limit. Negation follows the closed-world convention of RuleTaker: "X is not big" holds when "X is big" cannot be derived. Rules that test a negation run only after the positive rules have settled (a simple stratification), so they do not fire too early.

Engine B – CounterBench: search plus forward chaining. Variables without causes are the unknown roots. I enumerate every true/false assignment of the roots (a complete search, capped at 2^18 worlds) and keep the worlds that agree with the observations. In each kept world I apply the intervention ("if not X instead of X") and propagate the cause rules forward to see whether the target occurs. Each world is weighted by its prior, and the answer is "yes" when the target occurs in more than half of the weighted worlds.

Engine C – first-order logic: two stages.

1. *Horn fast path.* If every premise is a ground fact or a rule of the form $\forall$`x (A(x)` $\wedge$ `B(x)` $\rightarrow$ `C(x))`, and the query is a ground atom, I use forward chaining with unification over predicate–argument tuples. For such rules this is exact and takes well under a millisecond.

2. *General case: resolution by refutation with DPLL.* For anything with $\vee$, $\oplus$, $\leftrightarrow$, $\neg$ or $\exists$, I add the negated query to the premises and check whether the set is impossible. The steps are: convert to negation normal form, rename large sub-formulas inside $\oplus$ and $\leftrightarrow$ to stop blow-up, Skolemize $\exists$, turn the result into clauses (CNF), ground the variables over the constants in the problem, and run DPLL. DPLL is a depth-first search with unit propagation that branches on the shortest open clause. If no model exists, the query is entailed (True); otherwise the answer is False.

For a True/False label I only need the entailment check, so the engine runs one refutation per question instead of two. The second check (for a three-way True/False/Uncertain label) is kept only for the FOLIO numbers in the next section. Step and grounding limits protect the run time; if a limit is hit, the question gets the default answer.

Router. Each question goes to CounterBench first, then FOL, then English rules. An engine returns nothing when it cannot read a question, and the next engine is tried. This keeps the system general rather than tied to one dataset.

4. Experiments on the public datasets

On all 4,029 public questions the final system scores **92.03%**, and FOLIO jumps from the always-False baseline to 79–85% once it is read as logic.

Setup. I used RuleTaker (train, test and out-of-distribution splits), CounterBench (alpha and backdoor sets) and FOLIO (train and validation). RuleTaker and CounterBench are given in English.

FOLIO is given through its `premises-FOL` and `conclusion-FOL` columns, because the instructor announced that the evaluation input will be logic. FOLIO labels are mapped to two classes: True stays True, and False and Uncertain both become False. The public data was used only to test the system and to design the input reader; nothing was trained on it.

Dataset	**Questions**	**Engine**	**Accuracy (%)**
RuleTaker train	1,000	English rules	98.30
RuleTaker test	250	English rules	97.60
RuleTaker OOD (deeper proofs)	375	English rules	99.47
CounterBench alpha	1,000	CounterBench	95.50
CounterBench backdoor	200	CounterBench	90.00
FOLIO train (logic input)	1,001	FOL	79.92
FOLIO validation (logic input)	203	FOL	85.22
All	**4,029**		**92.03**

The whole run of 4,029 questions took **38.08 s**, or **9.45 ms per question**. Most of that time goes to FOLIO's resolution step, since the RuleTaker and CounterBench engines are very light. The router sent every question to the expected engine except one FOLIO train question, which the English engine picked up.

FOLIO with three labels. For completeness I also ran a second refutation to separate False from Uncertain. Three-label accuracy is lower, 66.93% on train and 72.41% on validation, because many True and False cases come out as Uncertain:

FOLIO train (true label → predicted)	**False**	**True**	**Uncertain**
False	144	25	120
True	7	230	151
Uncertain	9	19	296

The engine is careful rather than reckless: it rarely calls a True story False (7 of 388). When it cannot find a proof, it says Uncertain, which the two-label task maps to False.

What I tried along the way.

1. *English FOLIO without translation.* My regex parsers cannot read FOLIO's free English, so these questions fell to the default answer. That gave 62.04% on train and 65.02% on validation, almost exactly the always-False baseline (61.24% and 64.53%). This showed that the weak point was reading the English, not the reasoning.

2. *Gold FOL for FOLIO.* Feeding the given FOL formulas straight into Engine C raised FOLIO from about 62–65% to 79.92% (train) and 85.22% (validation). Many of the remaining errors I looked at are cases where a premise and the conclusion name the same idea with different predicate names, so the link between them is missing.

3. *Speed.* Adding the Horn fast path and dropping the second refutation (not needed for a True/False label) made the FOL engine faster in my tests, with no change in any prediction.

4. *Robustness tests.* I built 230 extra questions in nine files covering every notation (Unicode, ASCII, Prover9, Prolog, English, CounterBench) and file type (CSV, JSON, JSONL, TXT). The system answered all 230 correctly. These tests found one real bug: a Prover9 line starting with `-` (meaning "not") was read as a bullet point. I fixed it by treating `-` as a bullet only when a space follows.

5. Examples of reasoning traces

These traces come straight from the notebook (Section 11). Each shows the input, the steps the engine took, and the final answer.

Trace 1 – the example from the assignment (Engine A, English rules).

```
Bob is young. Bob is kind.
If someone is young, then they are smart.
If someone is smart and kind, then they are successful.
Query: Bob is successful.

If someone is young, then they are smart                =>  bob is smart        (?x = bob)
If someone is smart and kind, then they are successful  =>  bob is successful   (?x = bob)
query 'bob is successful' -> True
```

The first rule fires because `?x is young` unifies with `bob is young`. Its new fact, `bob is smart`, then lets the second rule fire. This matches the derivation in the assignment: Young(Bob) $\rightarrow$ Smart(Bob), then Smart(Bob) $\wedge$ Kind(Bob) $\rightarrow$ Successful(Bob).

Trace 2 – a False answer (Engine A, RuleTaker).

```
Facts: The wall dances gracefully. The river dances gracefully. The river becomes famous.
Rules: Assuming the river dances gracefully, we can say the river is fast. (+ 2 more)
Query: The sandwich dances gracefully.

Assuming the river dances gracefully, we can say the river is fast  =>  the river is fast
(no more rules can fire)
query 'the sandwich dances gracefully' -> False
```

Forward chaining reaches a fixed point without ever producing the query, so the answer is False.

Trace 3 – Horn logic (Engine C, fast path).

```
Young(bob)   Kind(bob)
∀x (Young(x) → Smart(x))
∀x (Smart(x) ∧ Kind(x) → Successful(x))
Query: Successful(bob)

4 premises parsed -> all Horn
forward chaining: Smart(bob), then Successful(bob)
query IS entailed -> True
```

Trace 4 – exclusive or (Engine C, resolution with DPLL).

```
∀x (Student(x) ⊕ Worker(x))
Student(ann)
Query: ¬Worker(ann)

add the negated query:          Worker(ann)
clauses after grounding:        [Student(ann) ∨ Worker(ann)], [¬Student(ann) ∨ ¬Worker(ann)],
                                [Student(ann)], [Worker(ann)]
DPLL unit propagation:          Student(ann)=T, Worker(ann)=T  ->  clause 2 is false
no model exists -> query IS entailed -> True
```

Xor is not a Horn rule, so the engine falls back to refutation. Assuming the opposite of the query leads to a contradiction, which proves it.

Trace 5 – "uncertain" becomes False (Engine C).

```
Rich(tom) ∨ Famous(tom)
Query: Famous(tom)

add ¬Famous(tom) -> DPLL finds a model (Rich(tom)=T, Famous(tom)=F)
query is NOT entailed -> False
```

The premises allow Tom to be rich instead of famous, so the query does not follow.

Trace 6 – counterfactual (Engine B, CounterBench).

```
We know that Ziklo and Blaf together cause not Trune, Trune causes Vork,
and Vork causes Lumbo. We observed Trune.
Query: Would Lumbo occur if not Ziklo instead of Ziklo?

roots [Blaf, Ziklo], observed {Trune: True}, change {Ziklo: False}
3 of 4 root worlds match the observation
force Ziklo = False -> Trune = True -> Vork = True -> Lumbo = True in every world
P(Lumbo | change) = 1.00 -> True
```

With Ziklo off, the AND that blocks Trune can never be true, so Trune, Vork and Lumbo all occur.

6. Limitations and conclusion

The system is strongest on logic input and on rule-style English, and weakest on long free-form English, which it does not try to read.

- **Free English.** Stories like FOLIO's raw text fall back to the default False. Since evaluation uses logic, this matters less, but the system is not a general English reader.
- **Mismatched names.** In logic input, `Young(x)` and `IsYouth(x)` are different predicates to the engine. If a dataset names one idea two ways, the link between premise and query is lost.
- **Closed-world negation.** English "not" follows the RuleTaker convention. A dataset that expects open-world answers would need `NL_CLOSED_WORLD = False`, which is one setting in the notebook.
- **Hard limits.** Very large FOL problems can hit the DPLL step limit and get the default answer.

To sum up, the system keeps the reasoning fully classical: forward chaining with unification for rules, complete search for counterfactuals, and resolution by refutation with DPLL for general logic. The same code runs on every input style without retraining, because nothing is learned; the answer for each question comes only from the facts and rules it contains.